Practical 11 (Post-lab):

Aim: Implementing a Virtual Memory System with Demand Paging where pages are loaded into memory only when needed, optimizing memory usage and minimizing page faults.

Code:

```c
#include <stdio.h>

int main() {
    int pages[50], frames[10];
    int n, f, i, j, k, flag, page_faults = 0, index = 0;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    // Initialize frames
    for(i = 0; i < f; i++) {
        frames[i] = -1;
    }

    printf("\nPage\tFrames\n");

    for(i = 0; i < n; i++) {
        flag = 0;

        // Check if page is already in frame
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                flag = 1;
                break;
            }
        }

        // If page not found → page fault
        if(flag == 0) {
            frames[index] = pages[i];
            index = (index + 1) % f;
            page_faults++;
        }

        // Print frames
        printf("%d\t", pages[i]);
        for(k = 0; k < f; k++) {
            if(frames[k] != -1)
                printf("%d ", frames[k]);
            else
                printf("- ");
        }
        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", page_faults);

    return 0;
}
```

Output:

```
athar@LAPTOP-8HI3V5MB MSYS ~
$ nano postlab.c

athar@LAPTOP-8HI3V5MB MSYS ~
$ gcc postlab.c -o postlab.exe

athar@LAPTOP-8HI3V5MB MSYS ~
$ ./postlab.exe

Demand Paging Simulation
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 1 -> 5 1 2
Page 2 -> 5 1 2
Page 3 -> 5 3 2
Page 4 -> 5 3 4
Page 5 -> 5 3 4

Total Page Faults = 9
Total Page Hits = 3
Hit Percentage = 25.00%
Fault Percentage = 75.00%

athar@LAPTOP-8HI3V5MB MSYS ~
$
```